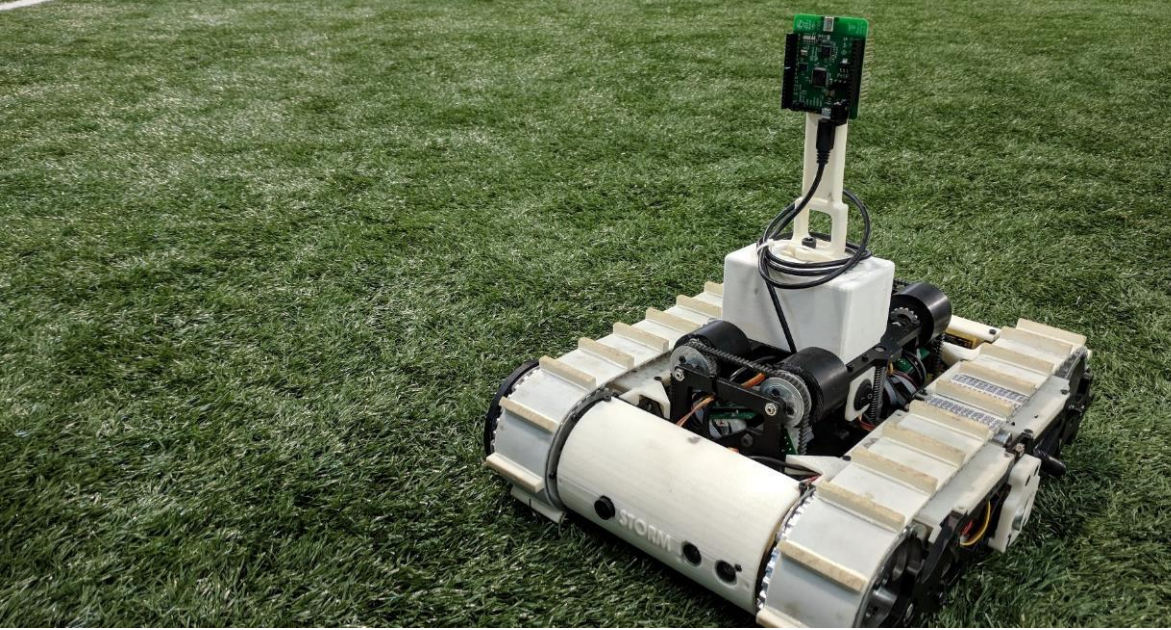




Path Planning for Pioneer P3-AT

Aadityanshu Abhinav (ME22B088)
Ajan Muthuraj (ME22B100)
Anuj Sreenivasan (ME22B106)



PROJECT DEFINITION

- **Problem:** Find an efficient path for a Pioneer 3-AT robot to navigate from its initial location to a goal in a dynamic environment
- The motion planning execution must satisfy the constraints of skid-steer wheeled robot dynamics
- This includes finding a path to the goal and maneuvering the vehicle along this path to avoid obstacles
- Maps will be given, but will change over time
- Algorithm must be adaptive to changes, without having to replan from scratch

We make various assumptions like we have perfect sensing, access to the global map at all times, the robot will not run out of battery, will get its global coordinates, moving obstacles have linear trajectories, etc, in order to focus on the motion planning aspects of this problem.



DELIVERABLES

Path Planning in Dynamic Environments (Simulation)

- Implement adaptive motion planning for a **Pioneer P3-AT** robot in **Gazebo**.
- Navigate through multiple simulated environments with varying layouts, static and dynamic obstacles with **different behaviours/trajectories**.

Performance Analysis & Optimization

- Evaluate **path efficiency**, **optimality**, and **adaptability** in dynamic scenarios.
- Compare different path-planning algorithms and refine for better real-time performance.

Real-World Deployment

- Validate the most optimal and efficient algorithm on a **physical robot**.
- Optimize for real-world uncertainties like sensor noise/drift and wheel slippage.

Extended deliverables:

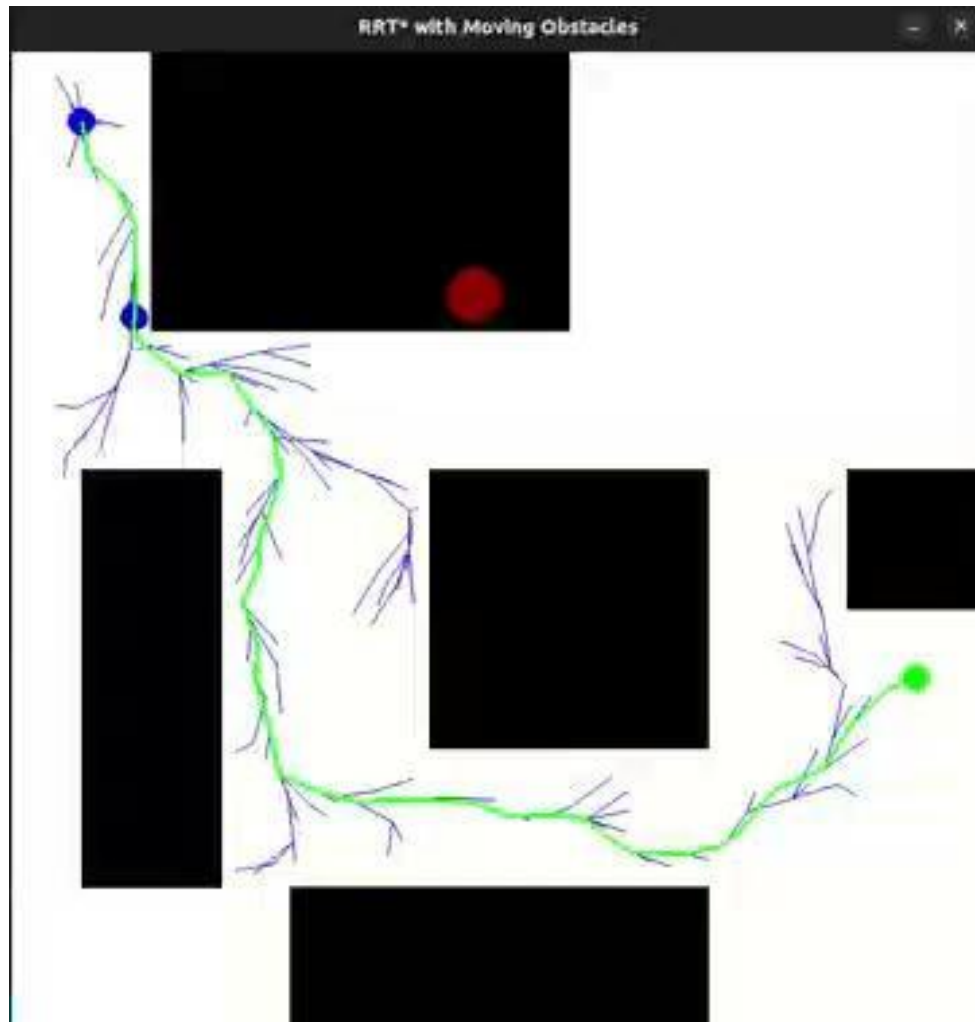
- Extend the simulation framework to handle **multiple autonomous robots**, each with different goal positions.
- Implement a decentralized/cooperative planning strategy to **avoid collisions and optimize efficiency**.

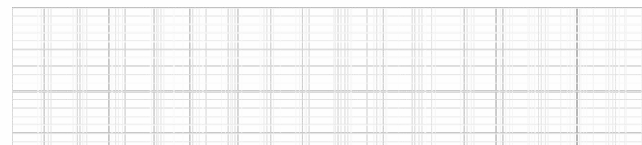
Enhanced RRT* Algorithm

- Compute an RRT* tree from start to goal
- Move the robot along the computed RRT* path
- Lookahead collision detection: Continuously monitor future segments of robot motion
- If an obstacle is predicted to cross the path ahead, trigger local replanning from a recent safe point
- Freeze the moving obstacle snapshot as a temporary static obstacle
- Rebuild a local RRT* tree to the goal, avoiding both static and "frozen" moving obstacles

Path planning with Adaptive RRT in PyGame

Dynamic environment (700 *
700 px), known map, point
robot





Alternate solutions explored.

- At first, there was a plan to use the known trajectories of the obstacle and the robot to predict any future collisions. However, this was not implemented successfully due to difficulties in tuning and it taking a long time to run (upwards of 10 minutes).
- An adaptive A* Algorithm with the same logic as above was also considered, but due to a lack of time it wasn't fleshed out.

Results

- The algorithm was run on a Pygame environment 50 times and the following parameters were measured:

Success rate: 45/50 (90.0%)

Avg replans per run: 1.34

Avg replan time: 0.44 seconds

Avg path length ratio (actual/ideal): 1.23

Near collisions: 121

The above indicates that the algorithm has an element of chance to it, where the collision is unlikely to happen only if the trajectory of the obstacle and the robot have no initial intersections.

Conclusions

- The algorithm was run on a Pygame environment 50 times and the following parameters were measured:

Success rate: 45/50 (90.0%)

Avg replans per run: 1.34

Avg replan time: 0.44 seconds

Avg path length ratio (actual/ideal): 1.23

Near collisions: 121

The above indicates that the algorithm has an element of chance to it, where the collision is unlikely to happen only if the trajectory of the obstacle and the robot have no initial intersections.